

오픈소스 개발자대회 · 결과보고서 초안

BakeNN

고정 모델 MCU를 위한 INT8 AOT 컴파일러

학습된 PyTorch FP32 모델과 대표 데이터를 받아
인터프리터 없는 정적 C11 추론 라이브러리를 생성

항목	내용
저장소	https://github.com/scienthoon/bakeNN
라이선스	Apache License 2.0
버전	BakeNN 0.1.0 (출품 준비 브랜치)
작성 기준일	2026년 8월 17일
작성자	scienthoon (GitHub)

공식 결과보고서 양식 수령 시 본문의 항목을 해당 서식에 이관

1. 출품작 개요

핵심 결과 BakeNN은 정적 batch-1 PyTorch FP32 모델을 대표 데이터로 PTQ하고, 검증된 INT8 실행 계획·SRAM 주소·커널 선택을 확정된 뒤 모델 전용 C11을 생성한다.

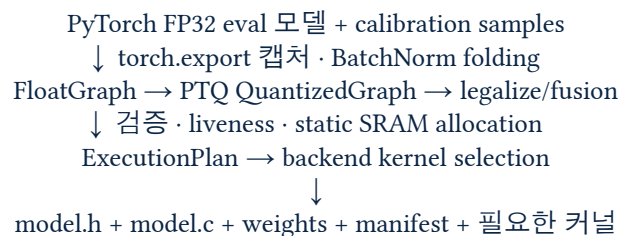
BakeNN은 MCU 펌웨어에 한두 개의 고정 모델을 포함하고 모델 변경 시 펌웨어도 함께 갱신하는 양산 제품을 대상으로 한다. 런타임에서 FlatBuffer를 해석하지 않고 호스트 컴파일 단계에서 그래프, 텐서 수명, 양자화 파라미터, 메모리 주소와 커널을 결정한다. 결과물은 `model.h/model.c/weights/manifest`와 필요한 커널 소스뿐이다.

정적 batch 1과 단일 공개 입력·출력 ABI는 의도적인 제품 제약이다. 내부 그래프는 residual, fan-out, concatenate, broadcast와 skip connection을 포함할 수 있지만, 동적 모델 로딩과 임의의 shape를 버림으로써 컴파일 시점 메모리 보증과 직접 호출 코드를 얻는다.

2. 개발 배경과 해결하려는 문제

- TFLite Micro는 넓은 연산 범위와 런타임 모델 교체 능력을 제공하지만, 고정 제품에서도 FlatBuffer, MicroInterpreter, op resolver, tensor planner와 런타임 메타데이터를 유지한다.
- 모델이 바뀌면 resolver 용량·연산 등록, operator version, TFLM 및 vendor kernel 버전의 호환성을 개발자가 함께 관리해야 한다.
- 모델 SRAM이 부족한 문제는 보드에서 AllocateTensors 실패로 늦게 발견될 수 있고, 최종 실행 순서를 감사하려면 모델 바이너리와 런타임 여러 계층을 함께 추적해야 한다.
- BakeNN은 범용 런타임 대신 고정 그래프를 강하게 정적 컴파일하여 이 조립·해석 비용을 호스트 단계로 이동한다.

3. 설계와 컴파일 파이프라인



1. 프론트엔드: torch.export의 정적 의미를 framework-neutral FloatGraph로 정규화하며 eval BatchNorm과 Dropout 등을 컴파일 시 제거·결합한다.
2. PTQ: 대표 입력으로 각 중간 activation 범위를 관측하고 per-tensor affine activation, per-output-channel symmetric weight, int32 bias를 결정한다.
3. 정수 IR: shape·layout·dtype·scale·zero point를 edge별로 고정하고 지원하지 않는 의미는 CompileError로 거절한다.

4. 정적 계획: 텐서 생존 구간을 계산해 activation buffer를 재사용하고 vendor scratch까지 하나의 정적 arena에 배치한다.
5. 코드 생성: 실행 순서와 주소가 상수인 C11 함수와 모델에 필요한 커널 소스만 방출한다.

4. INT8 수치 계약과 정확성

항목	BakeNN 0.1.0 계약
Activation	per-tensor affine int8 [-128, 127]
Weight	output-channel별 symmetric int8 [-127, 127], zero point 0
Bias/Accumulator	int32, host에서 accumulator overflow 상한 증명
Requantization	versioned Q31 double-round profile: bakenn.int8.v1
Padding	실수 0에 대응하는 input zero point 사용
검증 기준	Python integer reference ↔ portable/vendor C output byte-exact

변환 후 Python 정수 실행기와 생성 C는 같은 고정소수점 계약을 사용한다. Q31 multiplier-shift는 호스트에서 한 번 계산해 상수로 내보내며 타겟에서 float로 재계산하지 않는다. 지원 범위를 벗어난 vendor kernel은 AUTO에서 portable로 fallback하고 REQUIRE_OPTIMIZED에서는 이유와 함께 실패한다.

5. 구현 범위

분류	지원 내용	대표 모델/용도
핵심 연산	Conv2D/Depthwise/Linear, grouped Conv, Conv1D, grouped ConvTranspose2D	CNN, audio/time-series, 정적 U-Net
Activation	ReLU/Clamp, Sigmoid, HardSigmoid, HardSwish, SiLU	MobileNetV3, EfficientNet 계열
Pool/Reduction	Avg/Max 1D·2D, ReduceMean, global pooling	분류 head, temporal model
Tensor/Shape	Reshape/Flatten/Squeeze, static Slice/Crop, Concatenate, Pad, Resize	Residual, Dense, encoder-decoder
Elementwise	Add/Mul과 정적-rank broadcast, internal Requantize	SE block, residual connection
출력	BakeNN Q15-LUT Softmax	고정 class classifier

호스트 모델 게이트는 MobileNetV2/V3, EfficientNet-Lite-style, ResNet bottleneck, DenseNet-style concat, Inception, SqueezeNet Fire, compact U-Net, Conv1D classifier와 temporal residual 모델을 포함한다. TFLM 전체 연산 카탈로그와 동등하다는 의미는 아니다.

6. 타겟과 최적화 백엔드

타겟	실행 경로	현재 검증
범용 32-bit MCU	heap-free portable C11	GCC/Clang + ASan/UBSan
Cortex-M0+	portable C fallback	arm-none-eabi cross-link
Cortex-M4 DSP	SMLAD kernels + direct CMSIS-NN FC/Conv/DW/Pool	cross-ELF/disassembly + nRF52840 실측
RV32IMC	portable/generic C fallback	riscv GNU cross-link
ESP32	ESP-NN optimized Conv/DW + fallback	official C differential + ESP-IDF build
ESP32-S3	ESP-NN SIMD Conv/DW/FC/Pool + fallback	ANSI oracle differential + Xtensa ESP-IDF build
ESP32-C3	portable/generic fallback	ESP-IDF build

7. 검증 결과

전체 회귀 2026-08-17 제출 준비 브랜치에서 287 tests + 6 subtests가 통과했다. GitHub Actions는 Python 3.10/3.12 × GCC/Clang, PyTorch frontend/wheel, ARM/RISC-V cross-toolchain, ESP-IDF 3종을 검증한다.

- hand-calculated golden, malformed IR, qparam/overflow/alias 검증과 randomized differential를 함께 수행한다.
- 대표 생성 C는 strict C11, -Wall -Wextra -Werror -pedantic, ASan, UBSan으로 컴파일·실행한다.
- ESP32 optimized Conv/Depthwise는 10,000 random/edge tensor에서 Python reference와 byte mismatch 0이었다.
- ESP32-S3 Conv/Depthwise/FC/Pool wrapper는 graph별 10,000 tensor를 official ANSI oracle 경유로 비교해 mismatch 0이었다.
- wheel에는 pinned CMSIS-NN/ESP-NN source와 license/provenance가 포함되며 fresh venv 설치 후 모델 컴파일을 재검증했다.

8. 실제 학습 모델 PTQ 결과

trained MNIST full model 고정 seed로 60,000장을 4 epoch 학습한 CNN은 FP32 96.92%, 생성 C INT8 97.02%를 기록했다. 전체 10,000장 추론과 1,280 output-byte Python↔C 비교를 수행했으며 mismatch는 0이었다.

증거	값
FP32 checkpoint logical SHA-256	733130916e926da785afc63da0786d7613b0db808c3482398ae84b41ab706840
Calibration corpus SHA-256	06ce2a1aaaa0f75e566b9494c17b3c1fce4063272d1ce4e879c24b868bbb4e29
Generated artifact-set SHA-256	cc59172a18cc8d752185ab76c4a5a50c1bdccbcfb9abf8d8457a013e63d28f12
Physical corpus	100장 class-balanced, Python INT8 99/100 correct, expected output hash 고정

추가 model-family smoke로 MNIST 2종과 CIFAR-10 4종을 각각 전체 training split으로 1 epoch 학습하고, 100개 class-balanced 이미지로 calibration한 뒤 10,000개 test 이미지를 생성 C로 실행했다.

모델	Dataset	FP32	INT8 C	FP32-INT8
MNIST MLP	MNIST	92.69%	92.68%	+0.01 pp
MNIST CNN	MNIST	92.53%	92.52%	+0.01 pp
Bottleneck	CIFAR-10	20.20%	20.22%	-0.02 pp
Dense concat	CIFAR-10	18.83%	19.34%	-0.51 pp
Inception	CIFAR-10	18.64%	18.36%	+0.28 pp
Fire	CIFAR-10	18.59%	18.69%	-0.10 pp

여섯 실행 모두 Python INT8과 생성 C 비교에서 output byte mismatch가 0이었다. CIFAR-10 절대 정확도가 낮은 이유는 의도적으로 작은 모델을 단 1 epoch만 학습했기 때문이며, 본 실험은 SOTA 정확도가 아니라 실제 학습→calibration→PTQ→C 파이프라인과 양자화 차이를 검증한다.

9. 물리보드 성능 결과

증거 경계 이 절에는 실제 MCU에서 UART로 수집한 cycles·Flash·SRAM만 둔다. host 실행과 cross-build 결과는 다음 절에 분리하며 성능 수치로 사용하지 않는다.

9.1 nRF52840: TFLM/CMSIS-NN 비교

동일한 32→16→4 INT8 FC workload를 IoT-LAB nRF52840DK 64 MHz에서 동일 qparams·weights·bias·input bytes·output 의미로 비교했다. 8 warmups 후 101회를 측정했다.

Build	Median cycles	Flash	Static SRAM
BakeNN direct CMSIS-NN	3,786	20,920 B	8,540 B

Build	Median cycles	Flash	Static SRAM
TFLM + CMSIS-NN	5,418	69,640 B	11,040 B
BakeNN portable	8,706	20,764 B	8,540 B
TFLM reference	9,342	63,176 B	11,008 B

동일 FC workload 결과 BakeNN direct CMSIS-NN은 TFLM+CMSIS-NN 대비 cycles 30.1% 감소, linked Flash 70.0% 감소, linked static SRAM 22.6% 감소했다. 네 이미지의 output bytes와 FNV-1a 0x910c1fe2가 일치했다.

별도 $1 \times 4 \times 4 \times 1 \rightarrow 1 \times 4 \times 4 \times 2$ Conv2D에서 BakeNN portable은 TFLM reference 대비 24,610 대 27,441 cycles, 20,332 대 61,760 B Flash, 8,160 대 10,624 B static SRAM을 기록했다. 이는 CMSIS-NN Conv 비교가 아니며 모든 모델·MCU로 일반화하지 않는다.

9.2 ESP32: trained MobileNetV2-0.25

동일한 1-epoch CIFAR-10 checkpoint·calibration·INT8 graph·입력에서 ESP32-D0WDQ6 240 MHz, 8 warmups, 101회 측정으로 BakeNN portable, BakeNN+ESP-NN, TFLM+ESP-NN을 비교했다. 세 경로의 출력은 일치했다.

Path	Median	App binary	Linked DRAM
BakeNN portable C	285.546 ms	459,088 B	31,900 B
BakeNN + ESP-NN	97.685 ms	465,296 B	31,900 B
TFLM + ESP-NN	98.891 ms	665,504 B	95,332 B

이 artifact에서 BakeNN+ESP-NN은 TFLM+ESP-NN보다 latency가 1.22% 낮았고 app binary는 30.1%, linked DRAM은 66.5% 작았다. optimized kernel이 지배하는 CNN에서는 BakeNN의 큰 차이가 주로 메모리와 통합 단순성에서 나타났다.

10. 보드리스 cross-build 및 microTVM 비교

성능 주장 금지 cross-build는 타겟 ABI·컴파일러·링커·section size·undefined symbol을 검증한다. 실제 MCU cycles, cache, energy, stack high-water를 증명하지 않는다.

동일 trained MNIST checkpoint와 160장 calibration에서 만든 하나의 quantized contract를 BakeNN direct CMSIS-NN과 Apache TVM 0.16.0 AOT+USMP+CMSIS-NN으로 Cortex-M4에 cross-link했다. 두 생성 C는 각각 integer reference와 1,000/1,000 output bytes가 일치했다.

Path	Linked Flash	Static SRAM	Workspace	Physical cycles
BakeNN direct CMSIS-NN	12,088 B	4,864 B	4,064 B	미측정

Path	Linked Flash	Static SRAM	Workspace	Physical cycles
microTVM AOT+USMP+CMSIS-NN	17,456 B	4,862 B	4,064 B	미측정

이 ELF에서 BakeNN의 linked Flash는 microTVM보다 30.8% 작고 static SRAM은 사실상 같았다. trained MNIST ESP32 project도 cross-build와 corpus/hash 고정까지 완료했지만 UART transcript가 추가되기 전에는 물리 성능표에 넣지 않는다.

11. TFLM 대비 실용적 차별점

1. 외부 model interpreter를 MCU에 이식하지 않고 생성 C와 선택된 kernel만 MCU compiler로 빌드한다.
2. shape·padding·qparams·주소·실행 순서가 상수이므로 fusion, buffer reuse, packing과 전용 kernel 선택을 모델별로 수행한다.
3. constant, activation arena, scratch와 alignment를 호스트에서 계산해 Flash/SRAM budget 초과를 보드에 올리기 전 CI에서 거절한다.
4. 생성 C 호출 순서와 manifest를 직접 검토할 수 있어 FlatBuffer·resolver·planner·runtime을 함께 추적하는 것보다 펌웨어 감사 범위가 작다.
5. 실제 비교 과정에서 TFLM은 resolver 등록, Conv2D operator version, arena reservation, 구버전 CMSIS-NN wrapper 호환을 수동으로 맞춰야 했지만 BakeNN은 검증된 graph에서 kernel/source closure를 자동 확정했다.

대가로 BakeNN은 TFLM보다 연산 범위가 좁고, 모델을 바꾸려면 펌웨어를 다시 빌드해야 하며, 동적 shape·다중 공개 입출력·런타임 모델 교체를 지원하지 않는다. 이러한 요구가 중요한 제품에는 TFLM이 더 적합하다.

12. 오픈소스 구성과 재현 방법

항목	내용
Source	https://github.com/scienthoon/bakeNN
License	Apache-2.0; 상업 이용·수정·재배포 허용
CI	GCC/Clang, PyTorch, wheel, ARM/RISC-V, ESP-IDF
Community	CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, issue/PR templates
Evidence	물리 benchmarks/physical과 보드리스 benchmarks/cross_build를 분리
Clean room	scripts/verify_mnist_evidence.py; 외부 contributor PR gate
Demo	examples/mnist 및 examples/esp32s3_end_to_end

```
git clone https://github.com/scienthoon/bakeNN.git
cd bakeNN
python -m pip install -e '[test,torch]'
pytest -q
python examples/esp32s3_end_to_end/generate.py --output build/esp32s3_demo
```

13. 한계와 향후 계획

현재 한계	후속 작업
ESP32-S3 물리 cycle-energy 미측정	동일 모델 TFLM/ESP-NN 실보드 비교와 measured AUTO cost table
trained MNIST 물리 UART 미수집	준비된 100장 corpus로 ESP32 flash-cycles-stack transcript 고정
TFLM보다 좁은 operator surface	시장별 fixture 기반으로 detection/postprocess 등 선택 확장
PTQ만 구현, QAT fine-tuning 없음	PTQ 정확도가 부족한 모델을 위한 별도 QAT frontend
동적 shape·다중 공개 I/O 없음	핵심 제품 계약은 유지하고 필요한 정적 multi-ABI를 별도 검토
full application stack peak 미증명	최종 firmware ELF, stack watermark, RTOS/global 포함 측정

14. 제출 항목 상태

필수 항목	상태	비고
소스코드	준비	public GitHub, Apache-2.0
결과보고서 원본	준비	본 DOCX 초안; 공식 양식에 이관 필요
결과보고서 PDF	준비	본 문서 렌더 PDF
시연 절차	준비	3분 demo script와 ESP32-S3 one-command generator
시연영상 링크	미완료	화면 녹화·업로드 후 공식 양식과 본 문서에 링크 추가 필요
외부 clean-room PR	미완료	제3자가 fresh clone에서 1-command verifier 실행 후 JSON PR
trained MNIST physical	미완료	ESP32 UART transcript와 cycles/stack 결과 체크인
최종 release/tag	미완료	green PR merge 후 main에서 v0.1.0 tag/release

제출 전 필수 시연영상 링크를 추가하고 공식 주최측 양식에 본문을 이관해야 한다. 마감 전에 홈페이지 상태가 '제출 완료'로 바뀌고 자동 안내 메일이 도착했는지 모두 확인한다.

부록 A. 증거 파일

- benchmarks/tflm_compare/results/iotlab_447626_direct_cmsis_fc.md
- benchmarks/tflm_compare/results/iotlab_447626_direct_cmsis_fc_uart.txt
- benchmarks/tflm_compare/results/iotlab_447609_conv.json
- benchmarks/esp32/results/mobilenet_v2_025_cifar10_esp32_tflm_espnn.md
- benchmarks/microtvm_compare/results/mnist_cortex_m4_cross_build.json
- examples/mnist/evidence/mnist_evidence.json
- docs/COMPARISON.md
- docs/CLEAN_ROOM_REPRODUCTION.md
- examples/training_matrix/RESULTS.md
- benchmarks/RESULTS.md
- docs/P0_STATUS.md
- docs/P2_KERNEL_ARCHITECTURE.md
- docs/RELEASE_CHECKLIST.md